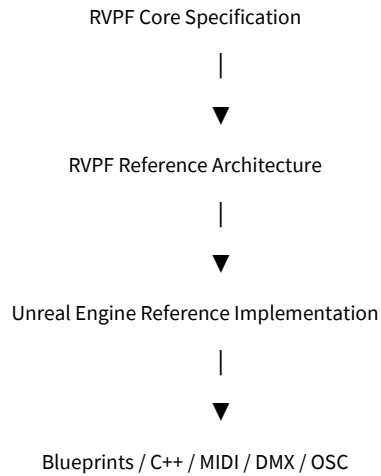


RVPF Core Specification



Document Metadata

Document ID	CS-001
Title	RVPF Core Specification
Version	0.1.
Status	Draft
Author	Olaf Erler
Maintainer	Olaf Erler
Created	2026-07-30
Last Modified	2026-07-30
Depends on	VSN-001
Referenced by	RA-001, Domain Models
Artifact Type	Core Specification

Purpose

This document defines the normative concepts, terminology and architectural principles of the Responsive Virtual Production Framework.

It serves as the authoritative source for all subsequent reference architectures, domain models and implementations.

Document Scope

This document defines

- the normative principles of the Responsive Virtual Production Framework
- the fundamental concepts governing semantic information processing
- the architectural constraints shared by all RVPF artifacts
- the mandatory requirements for all compliant RVPF implementations

This document explicitly does **not** define

- domain-specific terminology
- detailed domain models
- reference architectures
- implementation strategies
- software components
- hardware devices
- communication protocols
- target runtime environments

Related Artifacts	ID	Dokument	Beziehung
	VSN-001	Vision	Grundlage
	GLS-001	Glossar	Verwendet
	RA-001	Reference Architecture	Konkretisiert
	DM-CTX-001	Context Model	verwendet Begriff Context

Normative References	
ID	Beschreibung
VSN-001	RVPF Vision
GLS-001	Glossary
RA-001	Reference Architecture

Version	Datum	Änderung
0.1.	30.07.2026	Initial Draft

Architectural Motivation

Today, real-time applications form the foundation of modern virtual production systems. Cameras, tracking systems, lighting, audio, sensors, and user inputs continuously generate technical information that is processed immediately in real time.

In existing systems, this processing is primarily device- or implementation-specific. The meaning of a signal is often directly linked to its technical origin. This results in tightly coupled systems whose behavior is only limitedly reusable, portable, or extensible.

The Responsive Virtual Production Framework takes a different approach.

It consistently separates the technical origin of information from its domain-specific meaning. Technical information is initially treated as raw data. Only its interpretation within a production context leads to semantic events that determine the behavior of a real-time application.

This results in a domain-oriented architecture that describes production-related information independently of specific devices, communication protocols, or target platforms.

This Core Specification defines the fundamental concepts, terms, and principles of this architecture. It serves as the normative basis for all reference architectures, domain models, and implementations of the Responsive Virtual Production Framework.

This specification does not describe the implementation of a specific real-time environment, but rather the technical architecture of a universally applicable model for processing production-related information.

1. Purpose

Purpose of the Framework

The Responsive Virtual Production Framework (RVPF) is designed to extend virtual production systems with domain-oriented processing of production-related information.

It provides a unified middleware that captures signals from any technical source, interprets them within their production context, and converts them into semantic events. These events enable the responsive and interactive control of real-time environments, regardless of the underlying hardware or software.

The RVPF thus serves as the interface between physical or virtual signal sources and a real-time environment.

Normative Statement

First of all, an impulse has no technical meaning

Rationale

This separation prevents business logic from being tied to specific communication protocols.

2. Scope

Scope

The RVPF addresses the current practice of directly processing technical inputs within real-time applications.

In existing virtual production systems, signals are often processed directly by individual devices or software components. These inputs have neither a shared context nor a uniform semantic meaning.

The RVPF replaces this direct coupling with multi-layered processing.

- In this process,
- technical inputs are standardized,
- placed within their production context,
- state changes are detected,
- semantic events are generated,
- and subsequently passed on to StudioObjects via capabilities.

Rendering, simulation, and specific device implementations are not part of the framework. These remain within the respective target systems.

3. Terminology

The technical terms used in the framework are defined in the RVPF Glossary. This glossary serves as the authoritative semantic foundation for all models and specifications in the framework.

4. Core Principles

Principle 1

Impulses initially have no technical meaning.

An impulse represents nothing more than information that occurs over time. It is only through its interpretation within a context that it acquires semantic meaning.

Principle 2

Context determines meaning.

The meaning of a stimulus is determined solely by the current production context.

Principle 3

Events describe changes in state.

An event describes the semantic interpretation of a change within a context state.

Principle 4

Capabilities describe abilities, not devices.

The framework models functions using capabilities. Specific hardware merely represents implementations of these capabilities.

Principle 5

StudioObjects are the central domain entity.

All physical and virtual components of a production are described as StudioObjects.

Principle 6

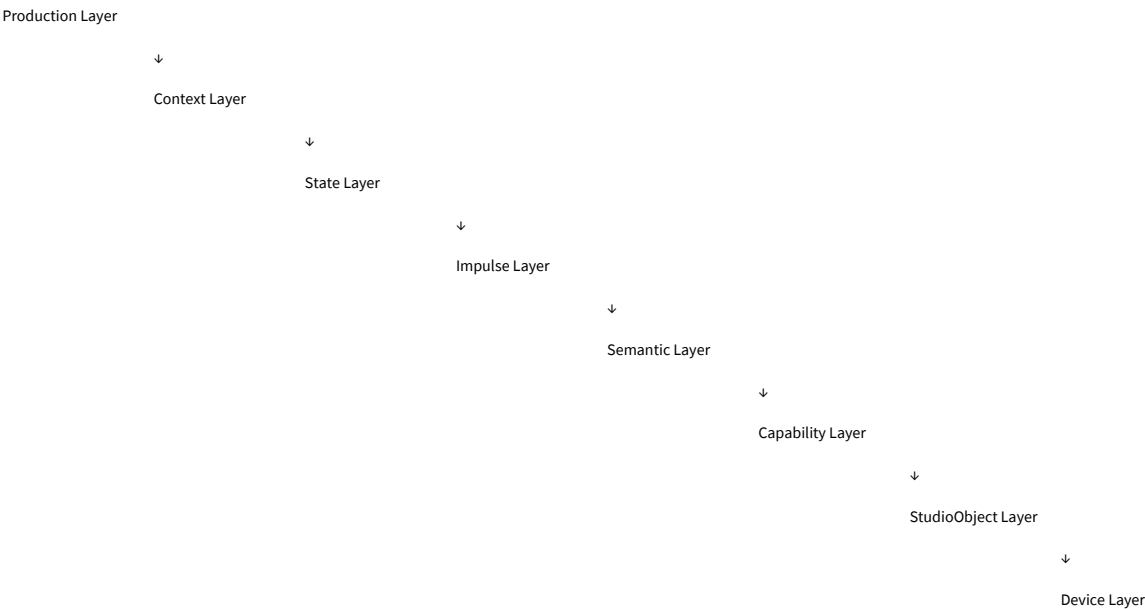
Implementations are interchangeable.

The processing of domain-specific information is independent of Unreal Engine, hardware, or communication protocols.

5. Reference Architecture

The reference architecture describes the logical processing of production-related information within the framework.

It consists of the following layers:



Each layer encapsulates a clearly defined responsibility and communicates exclusively through defined domain objects.

6. Domain Model

The RVPF domain consists of the following core objects:

- Production
- Context
- State
- Impulse
- Transition
- Event
- Capability
- StudioObject
- Device

There are defined relationships between these objects.

- Production has Context
- Context has State
- State is changed by Impulses
- State Transition generates Event
- Event activates Capability
- Capability is implemented by StudioObject
- StudioObject uses Device Adapter

The domain objects form the framework's central semantic model.

7. Behavior Model

The Behavior Model describes the flow of information within the framework.

A technical trigger is first detected.

The event is then applied to existing states within the current production context.

If the event changes at least one state, a state transition occurs.

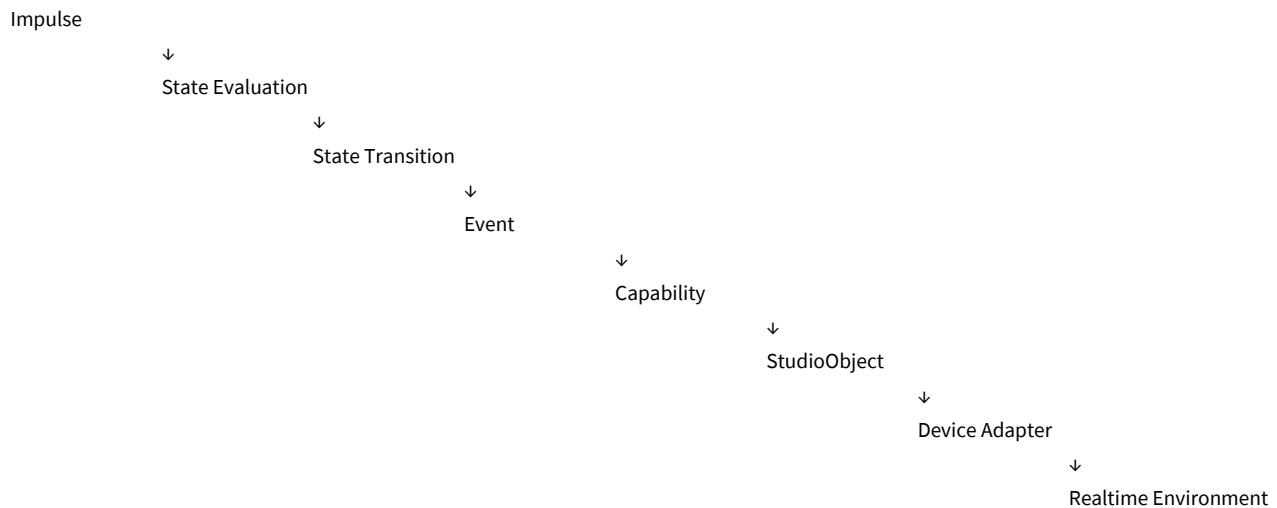
The semantic interpretation of this transition generates an event.

Events activate capabilities.

Capabilities are implemented by StudioObjects.

StudioObjects delegate the actual execution to the appropriate devices or software adapters.

This results in the following processing chain:



8. Implementation Mapping

The RVPF defines only the functional architecture for processing production-related information.

The specific technical implementation is carried out by adapters for the respective target platform.

For example:

RVPF	Unreal Engine
StudioObject	Actor / Blueprint
Capability	Actor Component / Interface
Event	Event Dispatcher
State	Variable
Context	Game State / Sequencer / Manager
Device Adapter	MIDI, DMX, OSC, Live Link
Realtime Environment	Unreal Engine

Other target platforms can implement the same specification without changing the framework's technical architecture.

9. Conformance

This chapter defines the minimum requirements that an architecture or implementation must meet to be considered compliant with the RVPF Core Specification.

Conformance Requirements

An RVPF-compliant implementation **shall**

- preserve the separation between technical impulses and semantic interpretation,
- interpret impulses within a production context,
- represent production entities as StudioObjects,
- describe behaviour through Capabilities instead of device-specific logic,
- preserve the semantic processing pipeline defined by this specification,
- separate domain logic from implementation-specific device communication,
- remain independent of concrete runtime technologies, communication protocols and hardware implementations.

Non-Conformance

An implementation is **not** RVPF-compliant if it

- processes technical signals directly as business events,
- couples devices directly to one another,
- integrates domain logic into communication adapters,
- bypasses the defined semantic processing chain,
- replaces StudioObjects with device-specific classes.